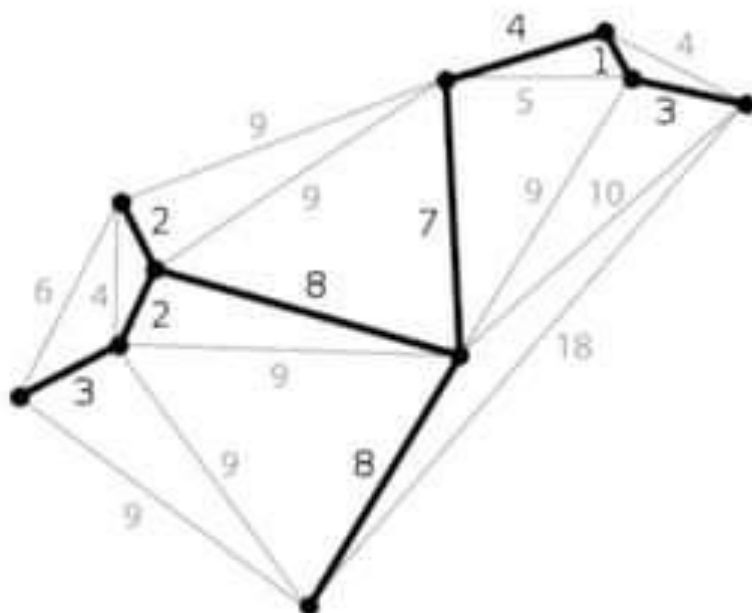


Minimum spanning trees

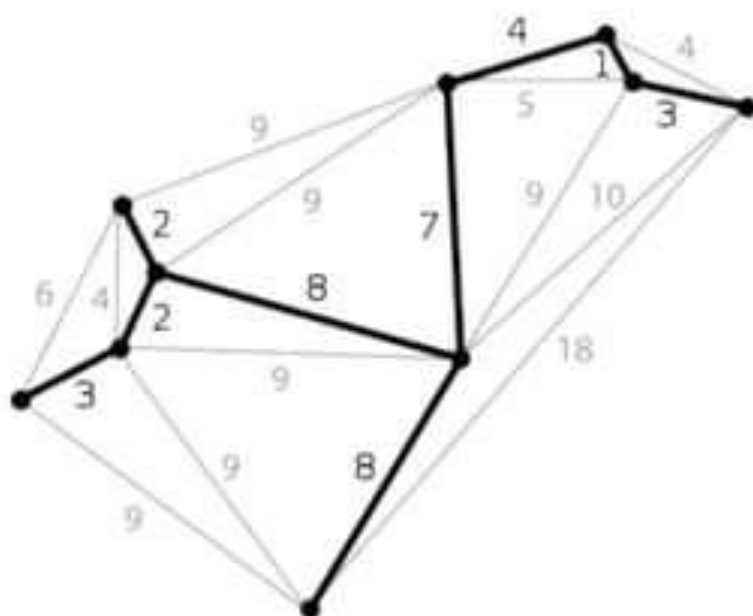
Ref: Wiki

- Definition (in an undirected graph):
 - A spanning tree that has the smallest possible total weight of edges



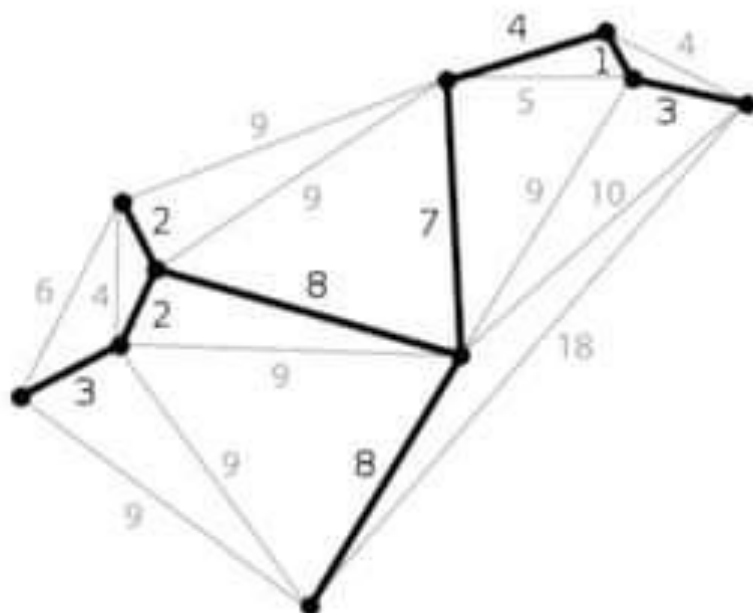
Minimum spanning trees

- Useful in broadcast:
 - Using a flood on the MST has the smallest possible cost on the network



Minimum spanning trees

- Useful in point to point routing:
 - Minimizes the max weight on the path between any two nodes



GHS Distributed MST Algorithm

Ref: NL

- By Gallagher, Humblet and Spira
- Each node knows its own edges and weights

GHS Distributed MST Algorithm

- Works in levels
- In level 0 each node is its own tree
- Each tree has a leader (leader id == tree id)
- At each level k:
 - All Leaders execute a convergecast to find the min weight boundary edge in its tree
 - It then broadcasts this in its tree so that the node that has the edge knows
 - This node informs the node on the other side, which informs its own leader

GHS Distributed MST Algorithm

- Observation 1:
 - We are possibly merging more than two trees at the same time
 - Problem: who is the leader of the new tree?
- Observation 2:
 - The merged tree is a tree of trees: it cannot have a cycle
 - We can assign a direction to each edge and each node (tree) has an outgoing edge
 - There must be a pair of nodes (trees) that select each-other (otherwise the merged tree is infinite)
 - We select the edge used to merge these two trees
 - Select the node with higher ID to be leader
 - The leader then broadcasts a message updating leader id at all nodes.

GHS Distributed MST Algorithm

- Complexity:
- The number of nodes at each level k tree is at least 2^k
- Since starting at size 1, the number of nodes in the smallest tree at least doubles every level
- Therefore, there are at most $O(\log n)$ levels

GHS Distributed MST Algorithm

- Complexity:
- At each level, at each tree, we use constant number of broadcasts and convergecasts
- Each level costs $O(n)$ time
- Total costs : $O(n \log n)$ time

GHS Distributed MST Algorithm

- Complexity:
- At each level, at each tree, we use constant number of broadcasts and convergecasts
- Each level costs $O(n)$ messages
- Total costs : $O(n \log n + |E|)$ messages

Distributed Spanning Tree Construction

Core idea of the GHS construction

- Start with a forest of individual one-node components
- Repeatedly identify a minimum-weight outgoing edge (MWOE) for each component
- Merge components through the selected MWOE
- Repeat level by level until only one component remains
- Each component has a representative leader UID

Level 0: the initial spanning forest

With n nodes, level 0 contains n separate spanning trees

Each component contains exactly one node and no selected edges

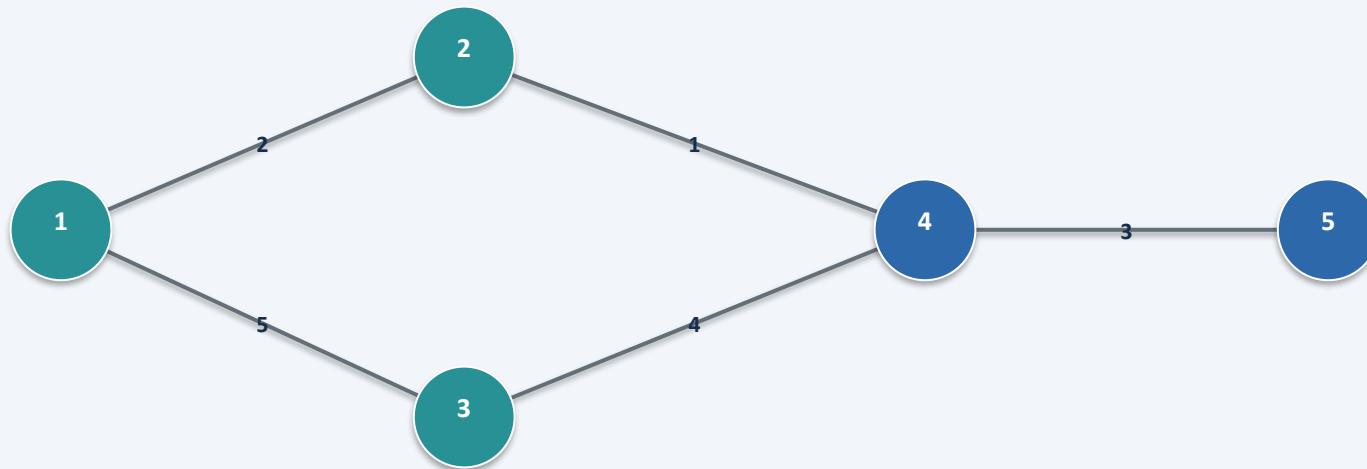
Every node knows its own UID and acts as the leader of its component

Nodes know their incident edges and their weights

Step 1: Search for the MWOE

- Each node participates in a search message broadcast within its current component to merge the two components
- This intra component broadcast is a BFS
- Each node examines incident edges and looks for edges leaving the component
- **M W O E** stands for **Minimum Weight Outgoing Edge**
- A test message to neighbors is used to determine their component identity

Step 2: Identify outgoing edges using component UUIDs



Each node asks its neighbors for their component leader UUID

Same UUID → same component → the edge is internal

Different UUID → different component → the edge is outgoing

Among its outgoing edges, each node keeps its minimum-weight candidate

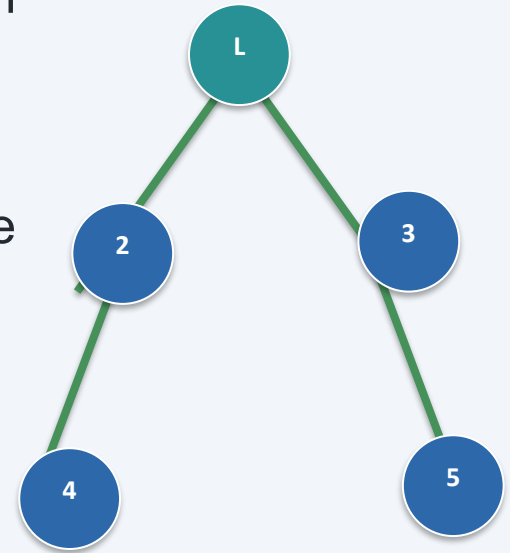
Step 3: Convergeicast candidate MWOEs to the leader

Each peripheral node sends its locally minimum outgoing edge toward the component leader

The convergeicast aggregates candidates from the component

The leader computes the global minimum

That edge becomes the component's MWOE for the current merge.



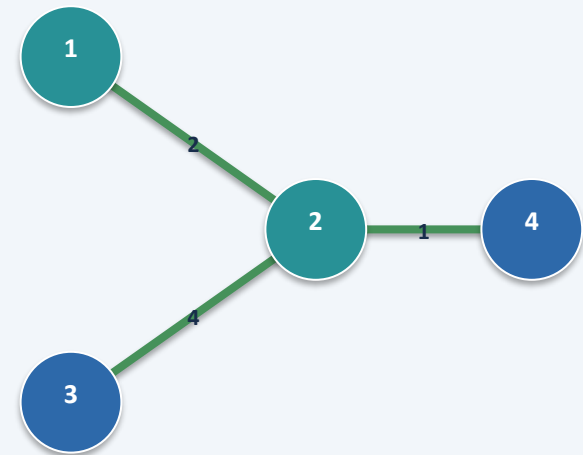
Step 4: Merge two components through the MWOE

The selected MWOE connects two different components

The current component leader communicates with the two endpoint nodes

Both endpoints mark the selected edge as part of the spanning structure

The two components are joined into one component



How is the new component leader selected?

After a merge, the two endpoint nodes know each other's UID

The node with the larger UID becomes the leader of the merged component

The new leader information is broadcast throughout the merged component



Why levels are essential

Components operate independently, so uncontrolled simultaneous merges could create conflicts

Two level-0 components merge to form a level-1 component

A level-0 component and a level-1 component produce a level-1 component

Equal level merges advance to the next level

Leveling imposes order and prevents conflicting activity

Termination: when is the MST complete?

Eventually the forest becomes one component

The leader starts another MWOE search

Every neighbor reports the same leader UID, so there is no edge to another component

The leader knows that the entire network is covered by one spanning tree and the construction can stop

Complexity analysis

At each level, $O(n)$ time is required for message propagation and processing

The number of levels is at most $O(\log n)$, because component sizes grow geometrically

Time complexity - $O(n \log n)$

Communication complexity: $O((n + e) \log n)$, where n is nodes and e is edges

Why the MST is useful beyond construction

Once an MST exists, it provides a communication backbone for distributed operations

Leaf nodes initiate a convergecast through the tree

Internal nodes combine information and forward it toward the remaining neighbor

The tree structure makes global aggregation simpler than operating over the original network

Leader election over the constructed tree

- Each node has a unique, comparable UID
- Leaves initiate convergecast messages
- An internal node waits for messages from descendants, combines the information, and forwards it
- If two neighboring nodes remain as candidates, the node with the higher UID becomes leader
- MST construction therefore provides a reusable structure for later distributed algorithms